

sumo_visualize.py

Spiegazione completa — cosa succede e perché

Progetto #2 — Automated Planning | UNICAL | Chiara, Elisa, Emanuele, Pierluigi

Parte 1 — La pipeline completa prima di arrivare a SUMO

SUMO non fa planning: visualizza soltanto un risultato già calcolato. Prima di poter eseguire *sumo_visualize.py* sono avvenute quattro fasi distinte.

Fase 1 — Scaricamento della mappa

`download_dublin_map.py` usa la libreria `osmnx` per interrogare OpenStreetMap e scaricare i dati stradali reali di Dublino: coordinate GPS degli incroci, nomi delle strade, limiti di velocità e sensi unici. Il risultato è un file `.osm` salvato in `osm_files/`.

Fase 2 — Codifica in PDDL+

`build_problems.py` legge il file `.osm` e costruisce il problema PDDL+. Ogni incrocio reale diventa un oggetto `location`. Ogni strada diventa il fatto (`road A B`), rispettando i sensi unici. Le distanze tra incroci vengono calcolate con la formula Haversine dalle coordinate GPS e diventano (`= (distance A B) 173`). Le velocità vengono prese dal tag `maxspeed` di OSM, convertite in `m/s`, e diventano (`= (speed A B) 8.33`). Risultato: un file `problem_*.pddl` che descrive fedelmente la mappa reale.

Fase 3 — ENHSP risolve il problema PDDL+

`run.py` lancia il planner ENHSP con il dominio e il problema scelto. ENHSP cerca la sequenza di azioni che porta dalla `START` alla `GOAL` minimizzando (`total-dist`). Il dominio definisce tre costrutti PDDL+: l'azione `start-move` (istantanea, avvia il movimento su una strada), il processo `driving` (continuo, fa avanzare `progress` proporzionalmente al tempo `#t`), e l'evento `arrive` (automatico, scatta quando `progress >= distance`). Il tempo nel piano è quindi continuo e fisico, non discreto. Il piano prodotto è una sequenza di strade con i tempi di partenza.

Fase 4 — Traduzione piano ENHSP → rotta SUMO

SUMO e PDDL+ usano identificatori diversi. Il piano ENHSP usa nomi come `liffey_st_upper`; SUMO usa ID numerici di archi stradali come `39994843`. La traduzione è fatta una volta sola con un BFS sul file `net.xml`: si prende la sequenza di nodi OSM dal piano, si trovano le junction SUMO corrispondenti (il `net.xml` incorpora gli ID OSM nei nomi delle junction), e si fa BFS nel grafo SUMO per trovare la sequenza connessa di archi. Il risultato — la lista di ID archi — è salvato direttamente in `sumo_visualize.py` e non cambia a ogni esecuzione: è fisso per ogni zona.

Parte 2 — Cosa succede quando esegui `sumo_visualize.py`

Lo script fa cinque cose in sequenza, poi apre `sumo-gui`. Non calcola nessun percorso: usa quello già trovato da ENHSP.

Step 1 — Legge la zona dalla riga di comando

```
zona = sys.argv[1] if len(sys.argv) > 1 else "piccola"
```

Controlla se hai scritto `piccola`, `media` o `grande` come argomento. Se non scrivi niente, usa `piccola` di default. Se scrivi una zona non valida, stampa un messaggio di errore ed esce.

Step 2 — Seleziona la configurazione per quella zona

```
cfg = CONFIGS[zona]
```

`CONFIGS` è un dizionario con tre voci, una per zona. Ogni voce contiene: il percorso al file `net.xml`, la sequenza di archi SUMO (`edges`) già calcolata da ENHSP, le coordinate del viewport (`zoom`, `x`, `y`) per centrare la vista sul punto di partenza, e le informazioni testuali (`distanza`, `tempo`, `nome start/goal`).

Step 3 — Scrive il file di route (`.rou.xml`)

```
<vType id="auto" maxSpeed="4.0" color="1,0,0" accel="1.5" decel="3.0"/>
<route id="piano_enhsp" edges="39994843 -1126998263#0 ..."/>
<vehicle id="veicolo_enhsp" type="auto" route="piano_enhsp" depart="1"/>
```

Questo file XML dice a SUMO tutto quello che serve sul veicolo e sul percorso. `vType` definisce come è fatta l'auto: accelerazione, frenata, velocità massima (4 m/s, lenta per vederla), colore rosso, forma berlina. `route` contiene la sequenza esatta di archi stradali nell'ordine corretto: l'auto li percorrerà uno dopo l'altro senza mai deviare. `vehicle` collega l'auto al percorso e dice quando parte (`depart=1` = al secondo 1).

Step 4 — Scrive i file di configurazione

```
<!-- gui_{zona}.xml -->
<viewport zoom="3000" x="663" y="749"/>

<!-- {zona}.sumocfg -->
<net-file value="piccola.net.xml"/>
<route-files value="piccola_piano.rou.xml"/>
<gui-settings-file value="gui_piccola.xml"/>
```

`gui_{zona}.xml` imposta la posizione iniziale della telecamera: `zoom` e coordinate `x,y` centrate sul punto di partenza del percorso. `{zona}.sumocfg` è il file principale che SUMO legge: dice quali file caricare (rete stradale, percorso, grafica) e la durata della simulazione (0-800 secondi).

Importazioni e argomento zona

```
import os, sys, subprocess

zona = sys.argv[1] if len(sys.argv) > 1 else "piccola"
if zona not in ("piccola", "media", "grande"):
    print("Uso: python sumo_visualize.py [piccola|media|grande]")
    sys.exit(1)
```

os/sys/subprocess sono moduli Python standard. sys.argv[1] legge il primo argomento da riga di comando. sys.exit(1) termina lo script con codice di errore se la zona non e' valida.

Dizionario CONFIGS

```
CONFIGS = {
    "piccola": {
        "net": os.path.join(BASE, "net_files", "piccola.net.xml"),
        "edges": "39994843 -1126998263#0 1478689539 ...",
        "zoom": 3000, "x": 663, "y": 749,
        "dist_m": 1570, "time_s": 194,
        "start": "Liffey Street Upper", "goal": "Aungier Street",
    },
    # ... media, grande
}
```

edges contiene la sequenza di archi SUMO trovata via BFS sul net.xml. Gli ID con il trattino (-317003249) indicano archi percorsi in direzione inversa. zoom/x/y centrano la telecamera sul punto di partenza. Il percorso e' fisso: non cambia a ogni esecuzione.

Scrittura del file di route

```
with open(ROU_PATH, "w") as f:
    f.write("""
<vType id="auto" accel="1.5" decel="3.0"
    maxSpeed="4.0" color="1,0,0" shape="passenger"/>
<route id="piano_enhsp" edges="{edges}"/>
<vehicle id="veicolo_enhsp" type="auto"
    route="piano_enhsp" depart="1"/>
""").format(edges=cfg["edges"])
```

maxSpeed=4.0 m/s e' bassa intenzionalmente: rende l'auto visibile e lenta. color=1,0,0 e' rosso in formato RGB normalizzato. depart=1 fa partire l'auto al secondo 1 della simulazione, non al secondo 0.

Ricerca di sumo-gui

```
def trova_sumo_bin(nome):
    sumo_home = os.environ.get("SUMO_HOME", "")
    candidati = []
    if sumo_home:
        candidati += [os.path.join(sumo_home, "bin", nome + ".exe")]
    for base in [r"C:\Program Files (x86)\Eclipse\Sumo",
                 r"C:\Program Files\Eclipse\Sumo"]:
        candidati.append(os.path.join(base, "bin", nome + ".exe"))
    for c in candidati:
        if os.path.exists(c): return c
    return None
```

Prima controlla la variabile d'ambiente SUMO_HOME, poi cerca nei percorsi standard di installazione SUMO su Windows. Se non trova nessun eseguibile restituisce None e lo script termina con errore.

Avvio di sumo-gui

```
subprocess.Popen([sumo_gui, "-c", CFG_PATH])
```

Popen (non run) apre sumo-gui come processo separato senza aspettare che finisca. Lo script Python termina subito dopo, ma sumo-gui rimane aperto. -c indica il file di configurazione da cui SUMO carica tutto il resto.

Nota: il piano PDDL+ (tempo teorico) e la simulazione SUMO (con semafori reali) danno tempi diversi di proposito. Il piano e' un lower bound ottimistico: velocita' costante, nessun semaforo, nessun traffico. SUMO valida il percorso in un ambiente piu' realistico, mostrando l'impatto dei semafori sul tempo effettivo di percorrenza.